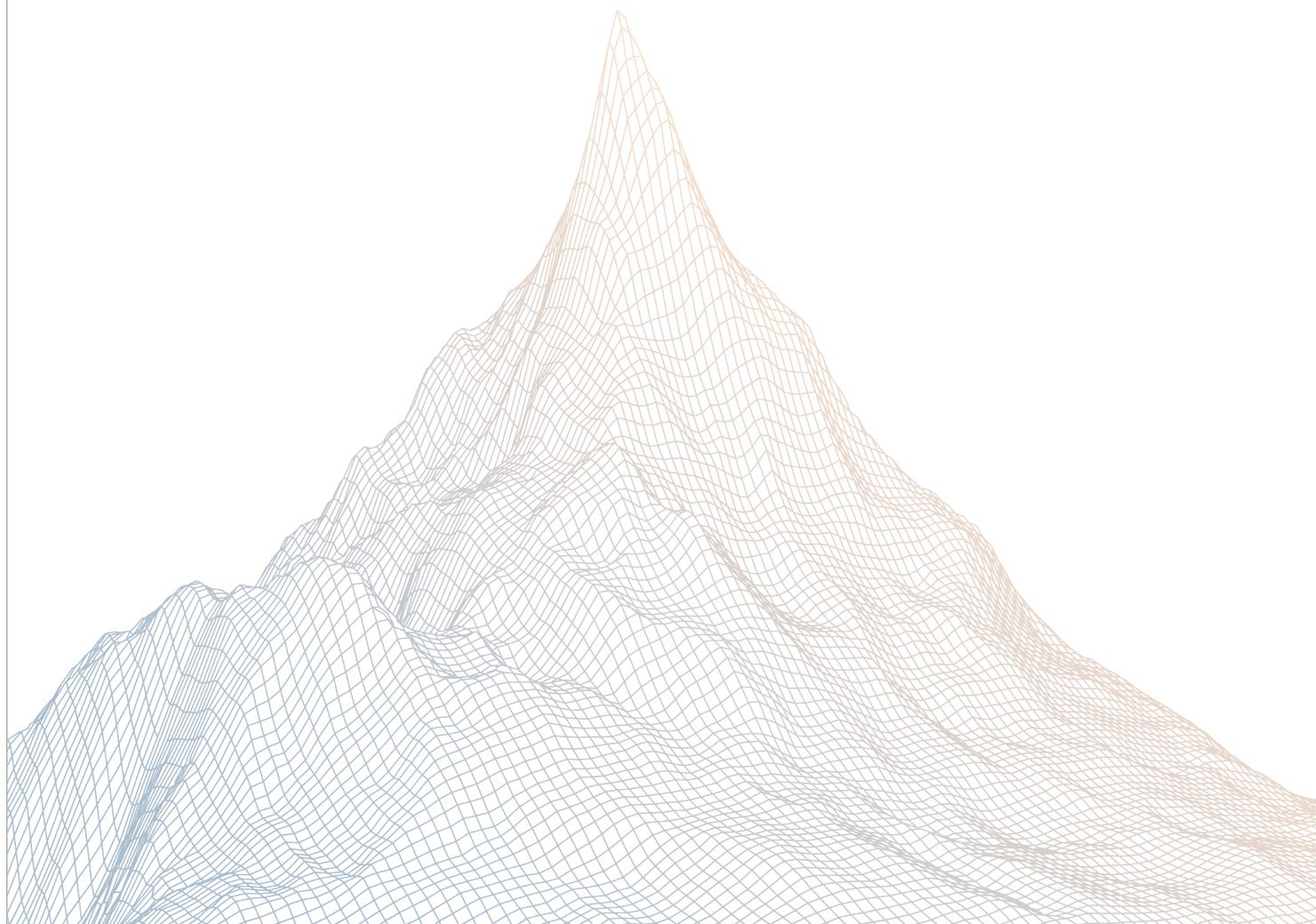


PAXGy

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: January 22nd to January 27th, 2026
AUDITED BY: HHK
ether_sky

The original Zenith report was titled Paxos Labs, a reference to the contracting entity. The report has been renamed as of 23 July 2026 but corresponds only to <https://github.com/paxoslabs/pxl-boring-vault/pull/237> audited on 22 January 2026

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About PAXGy	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	5
4	Findings	6
4.1	Low Risk	7
4.2	Informational	22

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About PAXGy

Boring Vaults are flexible vault contracts that allow for intricate strategies, secured by both onchain and offchain mechanisms.

2.2 Scope

The engagement involved a review of the following targets:

Target	pxl-boring-vault
---------------	------------------

Repository	https://github.com/paxoslabs/pxl-boring-vault/pull/237
-------------------	---

Commit Hash	75baa64bc289b1e70059abc0b499d86be640b6bd
--------------------	--

Files	Diff from 1a496469250619526c6511910511e147f9a08df0 src/base/Roles/WithdrawQueue.sol src/base/Roles/TellerWithMultiAssetSupport.sol src/base/Roles/CrossChain/CrossChainTellerBase.sol src/base/Roles/AccountantWithRateProviders.sol
--------------	--

2.3 Audit Timeline

January 22, 2026	Audit start
January 27, 2026	Audit end
January 31, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	8
Informational	2
Total Issues	10

3

Findings Summary

ID	Description	Status
L-1	Order processing can be blocked when withdrawing a large number of shares	Acknowledged
L-2	The EIP signature should also include the depositor address	Resolved
L-3	Bulk deposits are still possible even when the Teller is paused	Resolved
L-4	manageERC20 Can Transfer User Shares While Orders Are Pending	Acknowledged
L-5	No Mechanism to Invalidate Signed Withdrawal Orders	Resolved
L-6	Fee Reflection May Lag in Exchange Rate	Acknowledged
L-7	The processOrders function can revert in certain cases	Acknowledged
L-8	No Slippage Protection on Queued Withdrawals	Acknowledged
I-1	Unreachable Code in resetHighestExchangeRate	Resolved
I-2	AccountantWithRateProviders lacks bounds check inside constructor()	Resolved

4

Findings

4.1 Low Risk

A total of 8 low risk findings were identified.

[L-1] Order processing can be blocked when withdrawing a large number of shares

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [WithdrawQueue.sol](#)

Description:

BoringVault supports multiple deposit and withdrawal assets. If a user deposits a large amount in one asset but requests a withdrawal in a different asset, there's no guarantee that when the vault will have enough balance of the requested withdrawal asset. If the vault doesn't have sufficient balance of that withdrawal asset, `processOrders()` can revert.

```
function processOrders(uint256 ordersToProcess) public requiresAuth {
    ...
    else {
        BoringVault vault = BoringVault(payable(address(offerAsset)));
        uint256 expectedAssetsOut = tellerWithMultiAssetSupport.accountant()
            .getRateInQuoteSafe(ERC20(address(order.wantAsset)))
            .mulDivDown((order.amountOffer - feeAmount),
                10 ** vault.decimals());
        uint256 vaultBalanceOfWantAsset
            = order.wantAsset.balanceOf(address(vault));
        if (vaultBalanceOfWantAsset < expectedAssetsOut) {
            revert VaultInsufficientBalance(order.wantAsset,
                expectedAssetsOut, vaultBalanceOfWantAsset);
        }
        ...
    }
}
```

Recommendations:

There isn't a simple fix for this scenario. As a last resort, the owner can cancel the problematic order. That allows `processOrders()` to skip it, and the order is refunded to the receiver.

This can be addressed by improving documentation and/or adding some frontend safeguards.

Paxos Labs: Acknowledged. This is a reality of vaults and not in scope as we are concerned.

[L-2] The EIP signature should also include the depositor address

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [WithdrawQueue.sol](#)

Description:

If two different depositors submit orders with identical parameters, the computed hash will be the same.

```
bytes32 hash = keccak256(
    abi.encode(
        params.amountOffer,
        params.wantAsset,
        params.receiver,
        params.refundReceiver,
        params.signatureParams.deadline,
        params.signatureParams.approvalMethod,
        params.signatureParams.nonce,
        address(feeModule),
        block.chainid,
        address(this)
    )
);
```

In that case:

- The first depositor's transaction succeeds and marks the hash as used.
- The second depositor's transaction fails because `usedSignatureHashes[hash]` is already set to true.

Recommendations:

Including the depositor address in the signed data would prevent this collision.

Paxos Labs: Resolved with [@2b335c1faf ...](#)

Zenith: Verified.

[L-3] Bulk deposits are still possible even when the Teller is paused

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [TellerWithMultiAssetSupport.sol](#)

Description:

When the Teller is paused, deposits and withdrawals are blocked. However, bulk deposits are still allowed.

```
unction bulkDeposit(
    ERC20 depositAsset,
    uint256 depositAmount,
    uint256 minimumMint,
    address to
)
    external
    requiresAuth
    nonReentrant
    returns (uint256 shares)
{
    if (!isDepositSupported[depositAsset])
        revert TellerWithMultiAssetSupport__AssetNotSupported();

    shares = _erc20Deposit(depositAsset, depositAmount, minimumMint, to);
    emit BulkDeposit(address(depositAsset), depositAmount);
}
```

Recommendations:

```
unction bulkDeposit(
    ERC20 depositAsset,
    uint256 depositAmount,
```

```
uint256 minimumMint,  
address to  
)  
external  
requiresAuth  
nonReentrant  
returns (uint256 shares)  
{  
    if (isPaused) revert TellerWithMultiAssetSupport__Paused();  
    if (!isDepositSupported[depositAsset])  
        revert TellerWithMultiAssetSupport__AssetNotSupported();  
}
```

Paxos Labs: Resolved with [@83f3e482cd...](#)

Zenith: Verified.

[L-4] manageERC20 Can Transfer User Shares While Orders Are Pending

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [WithdrawQueue.sol#L243-L248](#)

Description:

The `manageERC20()` function allows admins to transfer any ERC20 token from the contract, including the `offerAsset` (vault shares) that users have deposited for pending withdrawal orders.

Unlike `setFeeModule()` and `setTellerWithMultiAssetSupport()` which require `totalSupply() ≠ 0` (empty queue), `manageERC20()` has no such check. An admin could accidentally or maliciously transfer user shares while orders are pending.

Recommendations:

Add a check to prevent transferring the `offerAsset` while orders are pending:

```
function manageERC20(IERC20 token, uint256 amount, address receiver)
    external requiresAuth {
    ...
    if (token == offerAsset && totalSupply() != 0) revert QueueMustBeEmpty();
    ...
}
```

Paxos Labs: Acknowledged.

[L-5] No Mechanism to Invalidate Signed Withdrawal Orders

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [WithdrawQueue.sol#L520-L535](#)

Description:

Users can submit withdrawal orders via signature using `submitOrder()` with `submitWithSignature = true`. The signature hash includes a nonce, but there is no function to invalidate a nonce or revoke a pending signature.

```
bytes32 hash = keccak256(
    abi.encode(
        params.amountOffer,
        params.wantAsset,
        params.receiver,
        params.refundReceiver,
        params.signatureParams.deadline,
        params.signatureParams.approvalMethod,
        params.signatureParams.nonce, // nonce included but can't be
        invalidated
        address(feeModule),
        block.chainid,
        address(this)
    )
);
```

Once a user signs a withdrawal order, they cannot cancel it before submission. The only protection is the deadline, but this may be set far in the future for convenience.

Attack scenario:

1. Alice signs withdrawal order with 1-hour deadline
2. Alice changes her mind or sign a new transaction with different amount
3. Alice has no way to invalidate the signature

4. Attacker can submit the old order anytime before deadline

Compare to EIP-2612 permits which use incrementing nonces (using any nonce invalidates all previous), or Permit2 which has `invalidateNonces()`.

Recommendations:

Add a function to allow users to invalidate signatures or add incremental nonces storage per user.

Paxos Labs: Resolved with [@2ad7c75f5b ...](#)

Zenith: Verified.

[L-6] Fee Reflection May Lag in Exchange Rate

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [AccountantWithRateProviders.sol#L341](#)
- [AccountantWithRateProviders.sol#L329-L345](#)

Description:

Performance and management fees accrue to `feesOwedInBase` on-chain but are reflected in `exchangeRate` with a lag (next oracle update). This creates a window where users can exit at a slightly favorable rate.

```
// Fees are calculated and added to feesOwedInBase
uint256 performanceFees = changeInAssets.mulDivDown(state.performanceFee,
    1e4);
newFeesOwedInBase += performanceFees;

// Exchange rate set without on-chain fee deduction
accountantState.exchangeRate = newExchangeRate;
```

Mitigating factors (per team response):

- Off-chain oracle factors `feesOwedInBase` into rate calculations on next update
- Team analysis shows ~0.005% variance with 24hr rate updates, even with 20% rate increase
- Cross-chain deployment constraints make on-chain fee deduction impractical (differing `totalSupply`, `block.timestamp` across chains would cause rate divergence)
- Effect smooths out over time; primarily impacts short-term users

Residual risk:

- Users withdrawing between rate update and next fee accrual avoid their proportional share of fees accumulated in that window
- Large holder exits right after rate update could avoid fees until next update (negligible at 0.005%)

Recommendations:

Clearly document the off-chain oracle's role in fee accounting and the trust assumptions this implies (oracle availability, correctness, and timely updates).

Proposed alternative: designate one chain as the canonical source of truth for rate calculations. Deposits/withdrawals on satellite chains would bridge to the canonical chain for processing, then bridge results back. This eliminates rate divergence but introduces bridge latency, additional bridge trust assumptions, and cross-chain message costs.

Paxos Labs: Acknowledged.

[L-7] The processOrders function can revert in certain cases

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [WithdrawQueue.sol](#)

Description:

When the Teller isn't paused, processOrders() is expected to keep processing orders and avoid reverting. To support that, if Teller.bulkWithdraw() would revert for a specific order, the order is refunded instead of reverting the entire batch. However, there's an edge case where processOrders() can still revert:

- There's no effective lower bound on minimumOrderSize, so an order's amountOffer can be extremely small.
- In that case, the computed asset output can round down to 0.

Example:

- minimumOrderSize = 0
- amountOffer = 2
- fee = 1 → net amount = 1
- rate = $9 * 10^{17}$
- 1 share = 10^{18}
- The computed expectedAssetsOut becomes 0 due to rounding.

```
function processOrders(uint256 ordersToProcess) public requiresAuth {
    {
        ...
        uint256 feeAmount = feeModule.calculateOfferFees(order.amountOffer,
            offerAsset, order.wantAsset, receiver);

        BoringVault vault = BoringVault(payable(address(offerAsset)));
        ▷ uint256 expectedAssetsOut = tellerWithMultiAssetSupport.accountant()
            .getRateInQuoteSafe(ERC20(address(order.wantAsset)))
            .mulDivDown((order.amountOffer - feeAmount),
                10 ** vault.decimals());
        uint256 vaultBalanceOfWantAsset
```

```

= order.wantAsset.balanceOf(address(vault));
    if (vaultBalanceOfWantAsset < expectedAssetsOut) {
        revert VaultInsufficientBalance(order.wantAsset,
expectedAssetsOut, vaultBalanceOfWantAsset);
    }

    try tellerWithMultiAssetSupport.bulkWithdraw(
        ERC20(address(order.wantAsset)), order.amountOffer - feeAmount,
0, receiver
    ) returns (
        uint256 assetsOut
    ) {
        if (assetsOut == 0) {
            revert InvalidAssetsOut();
        }
    } catch {
        orderAtQueueIndex[orderIndex].didOrderFailTransfer = true;
        // refresh the order from storage with the updated
didOrderFailTransfer
        order = orderAtQueueIndex[orderIndex];
        _refundOrder(order, orderIndex);
    }
    ...
}
}

```

In this scenario, `bulkWithdraw()` does not revert - it simply returns `0`. Since this is treated as a successful call rather than an error, the refund-on-revert logic doesn't trigger, and `processOrders()` ends up reverting when it encounters a zero-output withdrawal.

Recommendations:

```

function processOrders(uint256 ordersToProcess) public requiresAuth {
    {
        ...
        try tellerWithMultiAssetSupport.bulkWithdraw(
            ERC20(address(order.wantAsset)), order.amountOffer - feeAmount,
0, receiver
        ) returns (
            uint256 assetsOut
        ) {
            if (assetsOut == 0) {
                revert InvalidAssetsOut();
            }
        }
    }
}

```

```
        } catch {  
  
        }  
        ...  
    }  
}
```

Also it's better to always have a `minimumOrderSize` as it will also allow to block spamming the queue with very low / empty `withdraw` requests which could make processing the queue hard/slower especially on chains where gas is cheap.

Paxos Labs: Acknowledged. We will manually refund these orders and update the minimum if this ever happens (it likely will not if we are using minimums). Our primary concern here is preventing ourselves from being backed into a corner where we cannot unblock the queue. If it's possible to unblock in extreme edge cases, even with some admin effort that is acceptable.

[L-8] No Slippage Protection on Queued Withdrawals

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [WithdrawQueue.sol#L471-L483](#)
- [WithdrawQueue.sol#L71-L78](#)

Description:

The Order struct does not include a `minimumAssetsOut` parameter, and `bulkWithdraw()` is called with `0` for the minimum assets parameter:

Users submit orders expecting a certain exchange rate, but by the time the order is processed:

- Exchange rate may have dropped significantly
- Fee module output may have changed (if fee calculation depends on external factors)

Users have no way to specify a minimum acceptable output or cancel if conditions change unfavorably.

Recommendations:

Add `minimumAssetsOut` to the Order struct and `SubmitOrderParams`.

Then enforce it during processing:

```
if (assetsOut < order.minimumAssetsOut) {  
    // Mark for refund instead of processing at bad rate  
    _markOrderForRefund(orderIndex, false);  
    continue;  
}
```

Paxos Labs: Acknowledged. We do not want to use this for the queue. The assumption is users in the queue will be processed at the current market rate.

4.2 Informational

A total of 2 informational findings were identified.

[I-1] Unreachable Code in resetHighestExchangeRate

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [AccountantWithRateProviders.sol#L273-L275](#)

Description:

The resetHighestExchangeRate() function contains a check that can never be true under normal operation:

```
function resetHighestExchangeRate() external virtual requiresAuth {  
    ...  
    if (state.exchangeRate > state.highestExchangeRate) { // UNREACHABLE  
        revert AccountantWithRateProviders__ExchangeRateAlreadyHighest();  
    }  
    ...  
}
```

In updateExchangeRate(), whenever exchangeRate exceeds highestExchangeRate, the highestExchangeRate is immediately updated to match:

```
if (newExchangeRate > state.highestExchangeRate) {  
    // ... fee calculation ...  
    accountantState.highestExchangeRate = newExchangeRate;  
}  
accountantState.exchangeRate = newExchangeRate;
```

Therefore, exchangeRate > highestExchangeRate is never true after any updateExchangeRate() call. The check is dead code.

Recommendations:

Remove the unreachable check.

Paxos Labs: Resolved with [@0b9543302f ...](#)

Zenith: Verified. Kept the check but replaced with a more clear invariant assertion.

[I-2] AccountantWithRateProviders lacks bounds check inside constructor()

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [AccountantWithRateProviders.sol#126](#)

Description:

This issue was self reported by the Paxos team at the beginning of the engagement.

The constructor accepts several parameters that have bound checks in their respective setter functions but lacks these same validations during initialization:

- allowedExchangeRateChangeUpper
- allowedExchangeRateChangeLower
- minimumUpdateDelayInSeconds
- managementFee
- performanceFee

For example, `updatePerformanceFee()` enforces that `performanceFee` cannot exceed `1e4` (100%), but the constructor does not apply this check. Incorrect deployment configuration could result in fees exceeding 100% or other invalid state values with unforeseen consequences.

Recommendations:

Apply the same bound checks used in setter functions within the constructor.

Paxos Labs: Resolved with [@8ec7cef0f7...](#)

Zenith: Verified. Checks added to the `constructor()`.